LLM Efficiency — Homework

2026

Part 1 — KV Cache

The goal of caching the Key (K) and Value (V) states is to speedup the inference of autoregressive decoder like GPT.

During the practical, we started to adapt the code of minGPT from Karpathy in order to incorporate KV-caching. The goal of the first part of this homework is to finish the practical by completing `kv_cache.py` and running a small benchmark. We will only need the two main files `model.py` and `trainer.py` from Karpathy's repo. You will find these files in the `mingpt` folder (no changes are needed for these files).

Background

Using Named Tensor Notation, we write (see the paper by Chiang, Rush and Barak)

$$\text{Attention}\colon \mathbb{R}^{\mathsf{key}} \times \mathbb{R}^{\mathsf{seq}\times\mathsf{key}} \times \mathbb{R}^{\mathsf{seq}\times\mathsf{val}} \to \mathbb{R}^{\mathsf{val}}$$

$$\text{Attention}(Q,K,V) = \left(\underset{\mathsf{seq}}{\text{softmax}}\,\frac{Q \underset{\mathsf{key}}{\odot} K}{\sqrt{|\mathsf{key}|}}\right) \underset{\mathsf{seq}}{\odot} V.$$

During inference, when we compute the attention for the t-th token of a sequence, we get:

$$\text{Attention}\colon \mathbb{R}^{\mathsf{key}} \times \mathbb{R}^{\mathsf{seq}(t-b:t)\times\mathsf{key}} \times \mathbb{R}^{\mathsf{seq}(t-b:t)\times\mathsf{val}} \to \mathbb{R}^{\mathsf{val}}$$

$$\text{Attention}(Q_t,K_t,V_t) = \left(\underset{\mathsf{seq}}{\text{softmax}}\,\frac{Q_t \underset{\mathsf{key}}{\odot} K_t}{\sqrt{|\mathsf{key}|}}\right) \underset{\mathsf{seq}}{\odot} V_t,$$

where b is the size of a block and $t-b$ should be interpreted as $\max(t-b,0)$.

For the computation at time $t+1$, we see that the attention score depends on keys and values for all indices in $\mathsf{seq}(t-b+1:t+1)$. Since the keys and values for indices $\mathsf{seq}(t-b+1:t)$ were already computed at previous steps, we only need to compute the key and value for the new token at position $t+1$. This is exactly what the KV cache does!

It is perfectly fine to solve exercises 1-3 in the Jupyter notebook and then copy-paste your code in the python file `kv_cache.py`.

Exercise 1 — Modifying `CausalSelfAttention`

The class `CausalSelfAttention_kv` inherits the same architecture as Karpathy's original, but its `forward` method must accept and return a KV cache.

Signature:

```
def forward(self, x, kv_cache=None) -> (y, kv_cache):
```

The `kv_cache` is a list `[k, v]` where both tensors have shape `(B, seq_l, C)` — that is, they are stored **before** head splitting and transposition.

Exercise 2 — Modifying `Block`

Here is Karpathy's original `Block`:

```
class Block(nn.Module):
    """ an unassuming Transformer block """

    def __init__(self, config):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = nn.ModuleDict(dict(
            c_fc    = nn.Linear(config.n_embd, 4 * config.n_embd),
            c_proj  = nn.Linear(4 * config.n_embd, config.n_embd),
            act     = NewGELU(),
            dropout = nn.Dropout(config.resid_pdrop),
        ))
        m = self.mlp
        self.mlpf = lambda x: m.dropout(m.c_proj(m.act(m.c_fc(x)))) # MLP forward

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlpf(self.ln_2(x))
        return x
```

What to implement in `Block_kv.forward(self, x, kv_cache=None)`:

Thread the `kv_cache` argument through `self.attn`, collect the returned updated cache, and return both the block output `x` and the new `kv_cache`.

Exercise 3 — Modifying `GPT`

The `__init__` of `GPT_kv` has already been modified to use `Block_kv` instead of `Block`. You need to override `forward` and implement `generate_kv`.

Here is Karpathy's original `GPT.forward` and `GPT.generate` for reference:

```
    def forward(self, idx, targets=None):
        device = idx.device
        b, t = idx.size()
        assert t <= self.block_size, f"Cannot forward sequence of length {t}, block size is only {self.b
        pos = torch.arange(0, t, dtype=torch.long, device=device).unsqueeze(0) # shape (1, t)

        # forward the GPT model itself
        tok_emb = self.transformer.wte(idx) # token embeddings of shape (b, t, n_embd)
        pos_emb = self.transformer.wpe(pos) # position embeddings of shape (1, t, n_embd)
        x = self.transformer.drop(tok_emb + pos_emb)
        for block in self.transformer.h:
            x = block(x)
        x = self.transformer.ln_f(x)
        logits = self.lm_head(x)

        # if we are given some desired targets also calculate the loss
        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1), ignore_index=-1)
```

```python
        return logits, loss

    @torch.no_grad()
    def generate(self, idx, max_new_tokens, temperature=1.0, do_sample=False, top_k=None):
        """
        Take a conditioning sequence of indices idx (LongTensor of shape (b,t)) and complete
        the sequence max_new_tokens times, feeding the predictions back into the model each time.
        Most likely you'll want to make sure to be in model.eval() mode of operation for this.
        """
        for _ in range(max_new_tokens):
            # if the sequence context is growing too long we must crop it at block_size
            idx_cond = idx if idx.size(1) <= self.block_size else idx[:, -self.block_size:]
            # forward the model to get the logits for the index in the sequence
            logits, _ = self(idx_cond)
            # pluck the logits at the final step and scale by desired temperature
            logits = logits[:, -1, :] / temperature
            # optionally crop the logits to only the top k options
            if top_k is not None:
                v, _ = torch.topk(logits, top_k)
                logits[logits < v[:, [-1]]] = -float('Inf')
            # apply softmax to convert logits to (normalized) probabilities
            probs = F.softmax(logits, dim=-1)
            # either sample from the distribution or take the most likely element
            if do_sample:
                idx_next = torch.multinomial(probs, num_samples=1)
            else:
                _, idx_next = torch.topk(probs, k=1, dim=-1)
            # append sampled index to the running sequence and continue
            idx = torch.cat((idx, idx_next), dim=1)

        return idx
```

`GPT_kv.forward`

Signature:

```python
def forward(self, idx, targets=None, kv_cache=None, compute_first=False):
```

- `kv_cache` is a list of length `n_layer`, where each element is the `[k, v]` cache for that layer (or `None` if that layer has not yet been populated).
- `compute_first` is a flag used on the first call when a prompt is provided: when `True`, the full `idx` is processed even if a cache exists, in order to populate the cache from scratch.

`GPT_kv.generate_kv`

Adapt the original `generate` loop to use the KV cache for efficient generation:

Learning to sort

We use the demo to check that our code runs correctly. Run the `demo_sort.py` file (nothing to change in this file) to train the model on the sorting task and verify that `generate_kv` produces the same sequences as the original `generate`, then benchmark the two approaches.

Part 4 — Benchmark

Run the provided `benchmark.py` script, which compares the per-step latency of the baseline (no KV cache) against your `GPT_kv` implementation across two model sizes and several context lengths.

```
python benchmark.py
```

The script saves results to `benchmark_results.txt`. Your output must follow this exact format:

```
device: <cpu|cuda|mps>

gpt-mini  (X.XM params)
 Context T    no KV (ms)      KV (ms)   speedup
--------------------------------------------------
        64          X.XX          X.XX      X.XXx
       128          X.XX          X.XX      X.XXx
       256          X.XX          X.XX      X.XXx
       512          X.XX          X.XX      X.XXx
      1024          X.XX          X.XX      X.XXx
      1536          X.XX          X.XX      X.XXx

gpt2  (X.XM params)
 Context T    no KV (ms)      KV (ms)   speedup
--------------------------------------------------
        64          X.XX          X.XX      X.XXx
       128          X.XX          X.XX      X.XXx
       256          X.XX          X.XX      X.XXx
       512          X.XX          X.XX      X.XXx
      1024          X.XX          X.XX      X.XXx
      1536          X.XX          X.XX      X.XXx
```

Each row measures one generation step at the given context length T: - **no KV**: baseline `GPT.forward` re-encodes all T tokens (attention cost $O(T^2)$). - **KV**: `GPT_kv.forward` re-encodes only the one new token, with the previous $T-1$ token representations served from the cache (attention cost $O(T)$). - **speedup**: Ratio `no KV / KV`.

Part 2 — LoRA

The goal of this part is to adapt the code of minGPT from Karpathy to incorporate Low-Rank Adaptation (LoRA) for fine-tuning.

This blog post by Rajan Ghimire is a concise introduction to LoRA. The original paper is Hu et al., 2021.

It is perfectly fine to solve the exercises in the Jupyter notebook and then copy-paste your code in the python files `lora.py` and `demo_sort_lora.py`.

Background

Standard fine-tuning updates all $W \in \mathbb{R}^{d\times k}$ parameters of a pre-trained model, which is expensive when the model is large. LoRA freezes the pre-trained weights and injects a low-rank decomposition into each target layer:

$$h = Wx + \frac{\alpha}{r} BAx$$

where $A \in \mathbb{R}^{r\times k}$, $B \in \mathbb{R}^{d\times r}$, $r \ll \min(d, k)$, and α is a scaling constant. Only A and B are trained. At initialization, $B = 0$ so the adapter has no effect and training starts from the pre-trained checkpoint.

At inference time the correction $\frac{\alpha}{r}BA$ can be **merged** into W once, yielding $W' = W + \frac{\alpha}{r}BA$, so that LoRA adds zero overhead compared to the base model.

Exercise 1 — Building `LoRALinear`

`LoRALinear` subclasses `nn.Linear` and adds the LoRA adapter. All four methods below must be completed in `lora.py`.

`LoRALinear.reset_parameters_lora`

Initialise the LoRA matrices: - `lora_A`: Kaiming uniform with `a=sqrt(5)` (identical to `nn.Linear`'s weight init). - `lora_B`: zeros, so the adapter output is 0 at the start of training.

Also call `nn.Linear.reset_parameters(self)` to reset the base weight and bias.

`LoRALinear.forward`

The standard linear pass `x = Wx + b` is computed by `nn.Linear.forward`. When weights have **not** been merged and `lora_rank > 0`, add the LoRA correction:

$$h = Wx + b + \frac{\alpha}{r} \cdot B(Ax)$$

Use `F.linear` for both the A and B applications. When `has_weights_merged` is `True` the correction is already baked into `W`, so nothing extra is needed.

`LoRALinear.train`

When switching back to training mode (`mode=True`), **de-merge** if the weights were previously merged:

$$W \leftarrow W - \frac{\alpha}{r} \cdot BA$$

`LoRALinear.eval`

When switching to eval mode, **merge** the LoRA correction into `W` so that inference requires no extra computation:

$$W \leftarrow W + \frac{\alpha}{r} \cdot BA$$

Test: Call `.eval()` then `.train()` and verify the output is unchanged — merging then de-merging must be an exact round-trip.

Exercise 2 — Integrating LoRA into minGPT

`CausalSelfAttention_LoRA.__init__`

Call `super().__init__(config)` (which builds the standard attention layer), then **replace** `self.c_attn` and `self.c_proj` with `LoRALinear` instances, passing `lora_rank` and `lora_alpha` from `config`:

```
self.c_attn = LoRALinear(
    in_features=config.n_embd,
    out_features=3 * config.n_embd,
    lora_rank=config.lora_rank,
    lora_alpha=config.lora_alpha,
)
self.c_proj = LoRALinear(
    in_features=config.n_embd,
    out_features=config.n_embd,
    lora_rank=config.lora_rank,
    lora_alpha=config.lora_alpha,
)
```

`Block_LoRA` and `GPT_LoRA.__init__` are already provided: they substitute `Block_LoRA` (which uses `CausalSelfAttention_LoRA`) wherever the base classes used `Block`.

`GPT_LoRA.configure_optimizers`

When `lora_rank > 0`, skip the full decay/no-decay parameter grouping from the base `GPT` class and simply return a `AdamW` optimizer. Because `get_lora_model` has already frozen all non-LoRA parameters, only the LoRA matrices will receive gradient updates.

Exercise 3 — Learning to sort

We use the sorting task from Karpathy's demo as a testbed in the `demo_sort_lora.py` file.

Pre-training (code provided)

Train `GPT_LoRA` (with `lora_rank=8`, `lora_alpha=32`) on `SortDataset(split='train', length=6)` for 1000 iterations at learning rate `5e-4`. Evaluate on both the train and test splits using greedy decoding and report accuracy (fraction of sequences sorted correctly).

Distribution shift (code provided)

Evaluate the pre-trained model — without any fine-tuning — on `SortDataset(length=10)`. The model was trained on length-6 sequences, so performance on length-10 will be lower.

LoRA fine-tuning (to be completed)

Call `get_lora_model(model)` to freeze the pre-trained weights and then fine-tune only the LoRA parameters on `SortDataset(length=10)` for 2000 iterations. Re-evaluate on both splits and report the recovered accuracy.